variable provider is added after the JSON providers, the environment variable configuration value is used in `IConfiguration`.

The ability to collate configuration from multiple providers is handy on its own, but this design is especially useful for handling sensitive configuration values, such as connection strings and passwords. Section 10.2.3 shows how to deal with this problem, both locally on your development machine and on production servers.

10.2.3 Storing configuration secrets safely

As soon as you build a nontrivial app, you'll find that you need to store some sort of sensitive data as a setting somewhere. This data could be a password, a connection string, or an API key for a remote service, for example.

Storing these values in appsettings.json is generally a bad idea, as you should never commit secrets to source control; the number of secret API keys people have committed to GitHub is scary! Instead, it's much better to store these values outside your project folder, where they won't get committed accidentally.

You can do this in a few ways, but the easiest and most common approaches are to use environment variables for secrets on your production server and User Secrets locally. Neither approach is truly secure, in that neither stores values in an encrypted format. If your machine is compromised, attackers will be able to read the stored values because they're stored in plain text. These approaches are intended mainly to help you avoid committing secrets to source control.

TIP Azure Key Vault is a secure alternative, in that it stores the values encrypted in Azure, but you still need to use User Secrets and environment variables to store the Azure Key Vault connection details. See the documentation for instructions on using Azure Key Vault in your apps http://mng.bz/BR7v. Another popular option is Vault by Hashicorp (www.vaultproject.io), which can be run on-premises or in the cloud.